

DukanBook AI VoiceBot

Technical Guide

How the codebase works, including the local LLM (Ollama), RAG, and voice

1. Architecture at a glance

The app has three layers that talk over HTTP. The screens (Streamlit) call a backend (FastAPI), which stores data in SQLite and uses an AI brain to understand requests. The brain can run on a local model (Ollama) so the app needs no paid key.

```
Streamlit UI (screens)
| HTTP
FastAPI backend (app/main.py)
|-- brain -> understands the message (app/brain.py, app/llm.py)
|-- tools -> ledger / reminders / weather / maths (app/tools.py, app/general.py)
|-- RAG -> knowledge answers (app/rag.py + data/knowledge)
|-- voice -> speech<->text (app/voice.py)
`-- SQLite DB -> parties, transactions, reminders (app/db.py)

Local LLM: Ollama server (localhost:11434) <- the brain calls this, no key
```

2. Project structure

File	What it does
app/main.py	FastAPI app. Defines the endpoints: /chat, /voice/chat, /parties, /transactions, /reminders, /admin/seed, /admin/reset, /health.
app/brain.py	Decides how to answer: small-talk greetings, then the LLM, and an offline parser as a fallback.
app/llm.py	The LLM brain. Tool-calling loop over an OpenAI-compatible API (Ollama, or Gemini/Groq).
app/parser.py	Offline rule-based parser. Reads intent and entities when no LLM is available.
app/tools.py	Ledger actions the brain can call: add entry, get balance, list parties, search knowledge, reminders.
app/general.py	Weather (Open-Meteo) and a safe maths calculator.
app/rag.py	The knowledge engine: chunk, embed, and search the reference notes.
app/voice.py	Speech-to-text (faster-whisper) and text-to-speech (edge-tts).
app/db.py	SQLite schema and helpers (parties, transactions, reminders, balance).
app/config.py	Loads the .env file and reports which LLM / voice is available.
data/knowledge/	About 320 markdown reference notes (the RAG corpus).
ui/	Streamlit screens: Dashboard, Khata, AI Assistant, Reminders, plus shared helpers.

3. What happens when you send a message

Every typed or spoken message goes to the backend and through the brain. The brain tries the cheapest, safest path first.

- If it is a greeting or small talk (hi, hello, thanks, who are you), reply with a fixed friendly message. No LLM call.
- Otherwise, if an LLM is available, call it. The LLM decides which tool to use and fills in the details.
- If the LLM is not available (or fails), fall back to the offline rule-based parser.
- Either way, the same tools run, so the ledger behaviour is identical.

app/brain.py (simplified)

```
def respond(message, conn):
    if is_greeting(message): return canned_reply # small talk
    if config.has_llm():
        try: return llm.run(message, conn) # the AI brain
        except: pass # fall back if it fails
    return offline_respond(message, conn) # rule-based parser
```

4. The AI brain (tool-calling)

The brain is a chat model given a set of tools (function calling). It reads the request, returns which tool to call with what arguments, the backend runs the tool, feeds the result back, and the model writes the final reply in the user's language. This loop lives in app/llm.py.

```
messages = [system_prompt, user_message]
loop:
    reply = call_model(messages, tools) # OpenAI-compatible request
    if reply has no tool_calls: return reply.text
    for call in reply.tool_calls:
        result = run_tool(call.name, call.args) # e.g. add_ledger_entry(...)
        messages.append(tool result)
    # loop again so the model can answer using the result
```

The tools the model can call:

- add_ledger_entry(party, type, amount) — record a credit or debit
- get_party_balance(party) — how much someone owes
- list_all_parties() — all customers and suppliers
- search_knowledge(query) — the RAG knowledge lookup
- schedule_reminder(party, due_at) / list_reminders()
- get_weather(city) / calculate(expression)

5. The local LLM (Ollama)

To avoid paid API keys and rate limits, the brain runs on a local model through Ollama. Ollama runs a server on your machine and exposes an OpenAI-compatible endpoint, so the same code that would call a cloud API calls the local model instead.

- Ollama server address: `http://localhost:11434`, with an OpenAI-style path `/v1/chat/completions`.
- Model used: `qwen3:8b`. It supports tool-calling and handles Hindi / English / Hinglish well.
- No API key and no rate limits. It runs offline on the laptop.
- Set in the `.env` file as `OLLAMA_MODEL=qwen3:8b`. When this is set, the brain uses Ollama.

How the code chooses the backend (`app/llm.py`)

```
if OLLAMA_MODEL is set:
    url = http://localhost:11434/v1/chat/completions # local, no auth header
    model = OLLAMA_MODEL # qwen3:8b
else:
    url = Gemini / Groq OpenAI-compatible endpoint
    headers['Authorization'] = 'Bearer ' + API_KEY
```

One small detail: some local models (like `qwen3`) add hidden reasoning between tags. The code strips those out before showing the reply, so the user only sees the clean answer.

Why `qwen3:8b`

Among the local models tested, `qwen3:8b` was the one that returned proper structured tool calls and read Hinglish correctly (for example, it understood that 'Sam ke account me 2 hazar add karo' means add a credit of 2000 to Sam). `gemma3` did not support tools, and the `qwen coder` model returned the call as plain text instead of a structured tool call. A smaller model such as `qwen2.5:3b` is faster if speed matters.

6. The offline fallback (`app/parser.py`)

If no LLM is available, a rule-based parser handles the common commands. It detects the intent (`add` / `balance` / `list`) and extracts the person's name and the amount using patterns. It is limited compared to the LLM, but it keeps the app usable with zero dependencies.

7. The knowledge engine (RAG)

RAG (Retrieval Augmented Generation) lets the assistant answer GST, tax and business questions from real reference material instead of guessing. It lives in `app/rag.py` and the `data/knowledge` folder.

Build the index (once)

- Read the markdown notes in `data/knowledge` and split them into small paragraph chunks (about 320 chunks).
- Turn each chunk into an embedding (a vector of numbers that captures meaning) using `fastembed` with the `intfloat/multilingual-e5-large` model (1024 dimensions, with query/passage prefixes).
- Store the chunk text and its embedding in the `kb_chunk` table.

Answer a question (each time)

- Embed the question with the same model.
- Compare it against all stored chunks using cosine similarity and take the top 5 nearest.
- Hand those chunks to the brain, which answers only from them. If nothing matches, it says it is not sure.

The knowledge notes are curated from official sources (Income Tax Department, GST portal, MSME, MUDRA, PM SVANidhi, FSSAI, NPCI) with the source written in each file.

8. Voice (`app/voice.py`)

- Speech to text: `faster-whisper`, an offline version of OpenAI Whisper. Runs on the computer, no key. Model size is set by `WHISPER_MODEL` (default 'small'; 'medium' is more accurate).
- Text to speech: `edge-tts`, neural Indian voices, no key (needs internet).
- The `/voice/chat` endpoint ties it together: audio in -> transcribe -> brain -> reply -> speak back.

9. Database (`app/db.py`)

A single SQLite file (`dukanbook.db`) with four tables: `party`, `transaction`, `reminder`, and `kb_chunk` (the knowledge index). A party's balance is the sum of credits minus the sum of debits; a positive balance means the party still owes the shopkeeper.

10. Configuration and running

The .env file

```
OLLAMA_MODEL=qwen3:8b # use the local model (no key). Remove to use a cloud key.  
# GEMINI_API_KEY=... # optional: cloud fallback  
# GROQ_API_KEY=... # optional: cloud fallback  
# WHISPER_MODEL=small # voice accuracy: tiny | base | small | medium
```

Run it (three things)

```
1. Ollama running with the model: ollama run qwen3:8b  
2. Backend: .venv\Scripts\python -m uvicorn app.main:app --port 8000  
3. Screens: .venv\Scripts\python -m streamlit run ui/streamlit_app.py  
Open http://localhost:8501
```

On the first run the knowledge index builds itself from the notes, which downloads the embedding model once and then works offline.

11. Technology summary

Part	Tool	Key?
AI brain	Ollama (qwen3:8b), local	No key
Knowledge embeddings	fastembed (multilingual-e5-large)	No key
Speech to text	faster-whisper	No key
Text to speech	edge-tts	No key
Backend	FastAPI	-
Database	SQLite	-
Screens	Streamlit	-

The whole system runs on a single laptop with no paid API key. Cloud models (Gemini or Groq) are supported as an option by adding their key, but the default is the local Ollama model.